



Automated Road Condition Monitoring System Video Segmentation Method

Submitted By:

Name	Reg. No	Index No.	Email
U P N M Pathirana	ICT/2023/046	6036	ict2023046@as.rjt.ac.lk
W D P P Witharana	ICT/2023/045	6035	ict2023045@as.rjt.ac.lk
H P D N D Ariyathilaka	ICT/2023/044	6034	ict2023044@as.rjt.ac.lk
J G A D Madhushani	ICT/2023/043	6033	ict2023043@as.rjt.ac.lk

1. Introduction

The **Automated Road Condition Monitoring System** is a computer-vision-based project specifically designed to identify and analyze road damages such as potholes, alligator cracks, and raveling. Following the successful extraction and enhancement of video frames in previous phases, this fourth submission focuses on the **Image Segmentation** stage.

1.1 Purpose of Image Segmentation in this Project

The primary purpose of segmentation in this system is to partition an enhanced digital image into multiple segments (sets of pixels) to simplify or change the representation of the image into something more meaningful. In the context of road monitoring, segmentation is used to isolate the **Region of Interest (ROI)** the actual road damage from the background pavement. By accurately grouping pixels that belong to a defect, the system can distinguish between healthy road surfaces and various types of degradation.

1.2 Why Segmentation is Required After Frame Enhancement

Segmentation is performed after the enhancement stage because raw video frames often suffer from environmental degradations like sensor noise, compression artifacts, and uneven illumination. As established in Submission 03, these issues introduce "false edges" that can lead to false-positive detections. The enhancement pipeline (using CLAHE, denoising, and sharpening) stabilizes crack and pothole boundaries, providing the high-quality edge detail necessary for the segmentation algorithm to accurately identify the borders of a defect without being distracted by background noise.

1.3 Importance of Accurate Isolation

Identifying and isolating road degradation regions with precision is critical for the safety and maintenance of transportation networks, particularly in Sri Lanka where poor road conditions are a major factor in accidents. Accurate segmentation ensures that the system does not just detect *that* damage exists, but precisely *where* it is and what its boundaries are. This precision is the foundation for the next logical step: assessing the severity of the damage to prioritize repairs.

1.4 Connection to Final Integrated Deployment

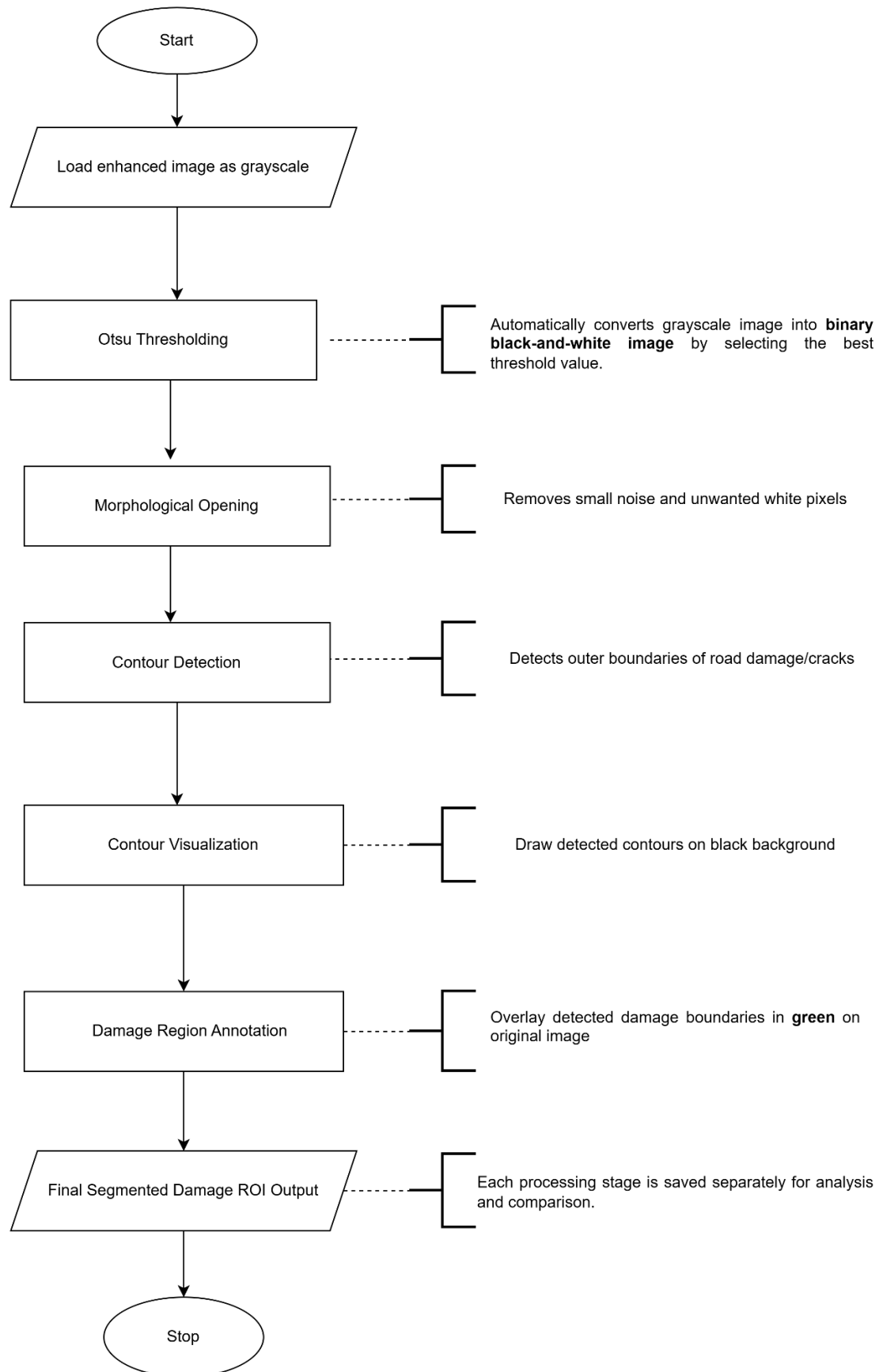
This segmentation stage serves as the final core processing block before the system is integrated into a complete, runnable program. The quality of the segmentation directly influences the reliability of the final automated output. Once segmented, these results are evaluated through both quantitative metrics (like accuracy and IOU) and qualitative visual inspections to ensure the system is robust enough for real-world deployment. This leads to the final goal: a fully integrated application where a user can input raw video and receive a processed output with all road damages clearly marked and classified

2.Proposed Frame Segmentation Pipeline

The segmentation pipeline was designed to process each enhanced road frame automatically and produce two clear outputs a binary mask showing the detected damage areas and a marked image showing the damage boundaries drawn on the original frame. The pipeline follows a sequential structure where the output of each stage becomes the direct input for the next, ensuring that every processing decision builds on the result of the previous one. This design keeps the pipeline simple, efficient, and easy to trace at each step.

The pipeline was implemented entirely in Python using the OpenCV library, which provides all the necessary image processing functions used across the stages. No manual parameter adjustment is needed during processing the threshold value is calculated automatically for each frame, and all outputs are saved and organized without any user input. The full sequence of processing stages is listed below, followed by a detailed explanation of each stage.

1. Load enhanced image as grayscale
2. Apply Otsu thresholding to generate a binary mask
3. Apply morphological opening to remove noise
4. Detect contours of damage regions
5. Draw contours on the original frame for visualization
6. Save the binary mask and the final marked output



❖ Step 1: Load Enhanced Image as grayscale

Before any segmentation can begin, the system needs to read the image from the folder and prepare it in the right format. Each enhanced frame is loaded into memory as a grayscale image, meaning it is represented as a single channel of pixel intensity values ranging from 0 to 255, where 0 is pure black and 255 is pure white. This is done using OpenCV's **imread** function with the grayscale flag.

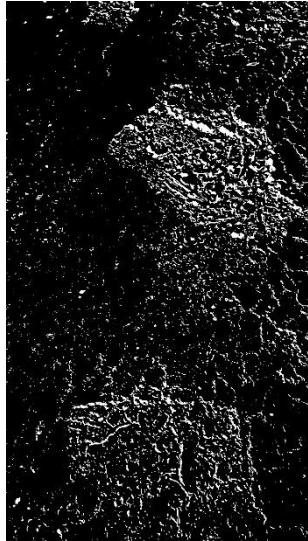
The reason grayscale is chosen here instead of a full color (BGR) image is straightforward. Road damage such as cracks, potholes, and surface wear appear as regions of lower intensity compared to the surrounding undamaged road surface. The actual color of the road does not help in finding these regions, what matters is how bright or dark each pixel is. By discarding the color channels and working only with intensity, the processing becomes simpler, faster, and more focused on what actually matters for detecting damage.

❖ Step 2: Otsu Thresholding

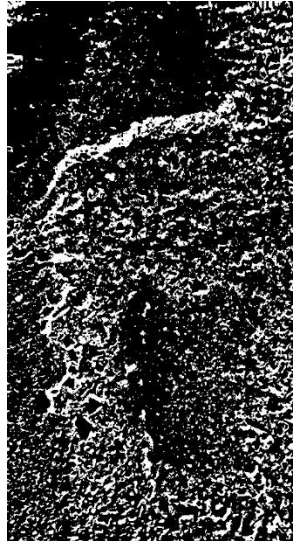
Once the grayscale image is ready, the next step is to convert it into a binary image an image where every pixel is either white (255) or black (0). This is done through a process called thresholding, where a threshold value is chosen and every pixel brighter than that value becomes white and every pixel darker becomes black, or the other way around.

The challenge is choosing the right threshold value. If it is too high, genuine damage regions get missed. If it is too low, normal road areas get incorrectly flagged as damage. To solve this automatically, the pipeline uses Otsu's thresholding method. Otsu's algorithm analyses the histogram of pixel intensities in the image and finds the threshold value that best separates the two groups of pixels dark regions and bright regions by minimizing the intra-class variance between them. This means the threshold is calculated fresh for each image based on its own content, with no manual tuning required.

In this pipeline, the inverse binary mode is also applied alongside Otsu. This means the darker pixels, which represent damage such as cracks and potholes, are turned white in the output, while the brighter road surface becomes black. The result is a binary mask where the white regions highlight the potential damage areas.



*Otsu thresholding
Potholes02*



*Otsu thresholding
Edge Cracking 02*

❖ Step 3: Morphological Opening

The binary mask produced by thresholding is not perfect. It often contains small isolated white pixels scattered across the image that do not correspond to any real damage. These appear because of subtle intensity variations in road texture, minor lighting inconsistencies, or sensor noise in the original video frames. If these are left in the mask, they will be detected as false damage regions in the next stage.

To clean the mask, a morphological operation called opening is applied. Morphological opening is a two-step process it first performs erosion, which shrinks all white regions by removing pixels from their edges, and then performs dilation, which expands the remaining white regions back to roughly their original size. The key effect is that very small isolated white spots, which are too small to survive the erosion step, are completely removed. Larger genuine damage regions lose a little from their edges during erosion but are largely restored during dilation, so they remain in the mask. A 3×3 rectangular structuring element (kernel) is used for this operation, which means the erosion and dilation are applied in a small neighborhood around each pixel.

- Purpose: Remove noise and tiny false-positive spots from the mask
- Kernel size: 3×3
- Iterations: 1
- Input: Noisy binary mask from thresholding
- Output: Clean binary mask with noise removed



*Morphological Opening
Transverse Cracking*



*Morphological Opening
Lane Marking Removal 01*

❖ Step 4: Contour Detection

With a clean binary mask available, the system now needs to identify the exact locations and boundaries of each damage region. This is done using contour detection. In image processing, a contour is a curve that traces the boundary between a white region and the black background in a binary image.

OpenCV's **findContours** function scans the binary mask and extracts these boundary curves for every white region it finds. The external retrieval mode is used, which means only the outermost boundary of each region is captured, and any inner holes are ignored. The contour approximation method used is the simple chain approximation, which stores only the corner and turning points of each contour rather than every single pixel along the boundary, making the data more compact. The output of this stage is a list of contours, where each contour represents the boundary of one detected damage region in the frame.



*Contour Detection
Transverse Cracking01*



*Contour Detection
Potholes03*

❖ Step 5: Contour Visualization

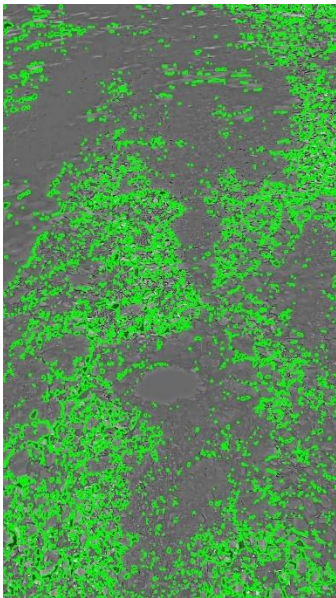
The contours detected in the previous stage are just coordinate data they are not yet visible. To make the results easy to inspect and evaluate, the contours are drawn on top of the original frame as a visual overlay.

First, the grayscale image is converted to a three-channel BGR color image using a color space conversion function. This step is necessary because drawing-colored lines requires a color image. Then, all detected contours are drawn onto this color image using OpenCV's **drawContours** function. The contours are drawn in green, specifically the BGR value (0, 255, 0), with a line thickness of 2 pixels. Green is chosen because it provides strong visual contrast against the grey tones of road surfaces, making the marked damage boundaries easy to see clearly. The result is the original frame with all detected damage regions outlined in green, giving an immediate visual confirmation of what the system has identified.

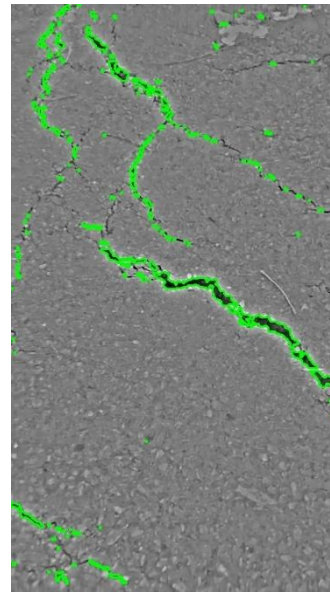
❖ Step 6: Damage Region Annotation and Final Segmented Output

In the final stage, the outputs of the pipeline are saved to disk in an organized way. Two files are saved for every processed image. The binary segmentation mask, which shows the detected damage as white regions on a black background, is saved to the Segmentation Masks output folder. The marked result image, which shows the original frame with green contour outlines drawn around each damage region, is saved to the Final Marked Detections output folder. Both folders are automatically created if they do not already exist, and the files are organized into sub-folders by degradation category to keep results structured.

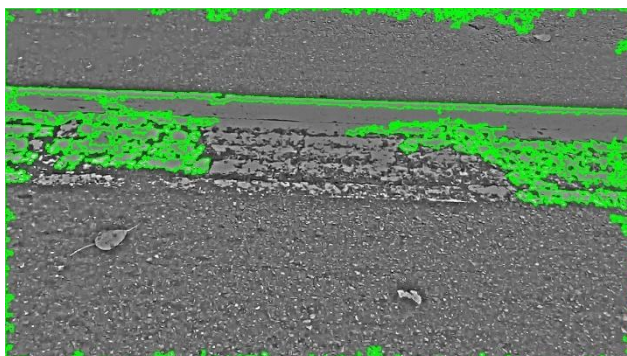
The green-outlined images represent the final segmented damage regions of interest (ROI) for each frame. These outputs serve two purposes they are used for visual and qualitative evaluation of how well the pipeline performed, and they feed into the final integrated system where enhancement and segmentation results are combined into a complete road damage detection output.



Output:
Raveling



Output:
*Transverse
Cracking*



Output:
Crossing Marking Removal

3. Experimental Setup and Results

3.1 Test Data Preparation

The test data used for segmentation was prepared from multiple road damage videos. Enhanced frames from the previous enhancement stage were used as input to the segmentation pipeline. Frames were selected to cover at least three different road degradation categories to ensure variety in testing.

Video ID	Frames Extracted	Frames Tested	Degradation Type	Lighting Condition
V001	66	21	Alligator Cracking01	[e.g. Daylight] Shadowed
V002	33	17	Transvers Cracking01	Daylight
V003	52	25	Crossing Marking Removal02	Daylight
V004	33	10	Raveling01	Shadowed + Sunlight
V005	28	13	Lane Marking Removal01	Daylight
V006	43	9	Potholes02	Daylight + Shadowed
V007	41	11	Edge Cracking02	Daylight
V008	70	30	Potholes03	Shadowed + Daylight
V009	96	9	Potholes01	Shadowed + Daylight

Table 1: Test Data Preparation Summary

4. Recommendation

4.1 Final Recommendation Based on Results

Based on the experimental results, the developed segmentation pipeline is suitable for detecting clearly visible road degradations such as deep cracks and potholes under standard daylight conditions. It is reliable enough for use in the final integrated system as a baseline, particularly for degradation types that show strong contrast with the surrounding road.

The pipeline is most effective for detecting: large cracks (longitudinal and transverse), potholes with dark interiors, and surface-level damage with strong contrast.

4.2 Critical Analysis of the Segmentation Method

Strengths: Otsu's thresholding is fully automatic it does not need manual threshold tuning for each image. The morphological opening step effectively removes noise and prevents small random spots from being flagged as damage. The contour drawing step makes results easy to visually verify. The overall pipeline is simple, fast, and straightforward to implement.

Limitations: The method relies entirely on pixel intensity differences. It cannot distinguish between damage and other dark features such as shadows, water reflections, or dark road markings. It treats every frame independently without using any context from neighboring frames.

4.3 Limitations Observed

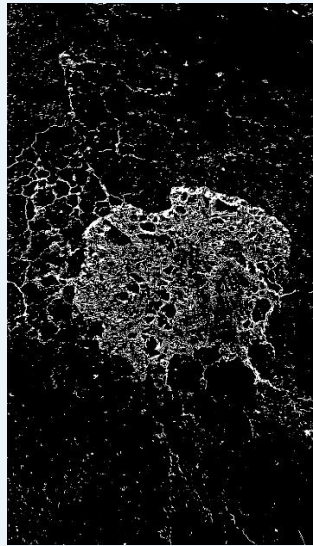
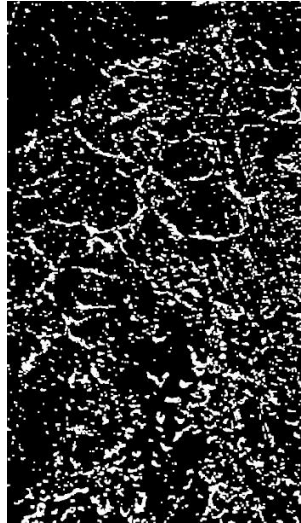
- Strong shadows create dark regions that can be mistaken for damage
- Uneven lighting across the frame causes inconsistent threshold performance
- Faded road markings produce false detections
- Similar texture between damaged and undamaged road areas makes separation difficult
- Motion blur reduces sharpness of damage edges, affecting contour accuracy
- Over-segmentation in textured road surfaces
- Under-segmentation of very thin cracks with low contrast

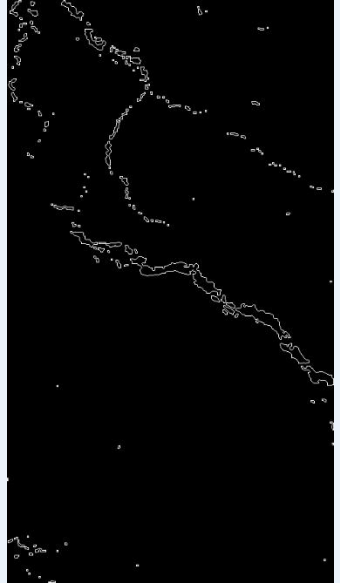
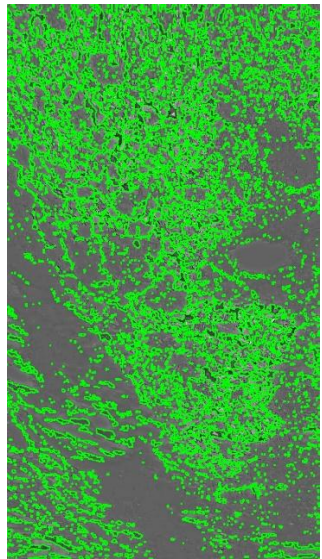
4.4 Suggestions for Improvement

- Use adaptive thresholding (e.g. Sauvola or local Otsu) instead of global Otsu to handle uneven illumination better
- Add contour area filtering to remove very small detected regions that are likely noise
- Apply temporal analysis across consecutive video frames to confirm damage across multiple frames before marking it
- Use more advanced segmentation methods such as watershed or deep learning-based models for more complex scenes
- Build an annotated ground truth dataset and use it to train or fine-tune the thresholding parameters per degradation type
- Improve the enhancement stage for shadow removal before segmentation

5. Team Contribution Planning and Individual Responsibilities

Each team member was assigned a specific road degradation type to segment and evaluate. The contribution of each member is listed in the table below.

Member Name (Reg / Index)	Segmentation Responsibility	Coding Contribution	Experimental Task	Evidence
ICT/2023/045 6035	Pot Holes 1	Otsu Thresholding Stage	Test on selected crack frames	
ICT/2023/046 6036	Alligator Cracking 01	Morphological Opening Stage	Test on selected crack frames	

Member Name (Reg / Index)	Segmentation Responsibility	Coding Contribution	Experimental Task	Evidence
ICT/2023/044 6034	Transverse Cracking 01	Contour Detection & Drawing	Test on frames	
ICT/2023/043 6033	Raveling 01	Output Saving & Folder Structure	Test on surface wear frames	

6. Final Integration Program

6.1 Integrated Processing Pipeline

The final integrated program combines the image enhancement stage developed in the previous submission with the segmentation pipeline described in this report. The full processing flow is:

- Load input video or folder of extracted frames
- Apply image enhancement (contrast improvement, noise reduction, etc.) to each frame
- Pass each enhanced frame through the segmentation pipeline (thresholding, morphological opening, contour detection)
- Save the binary mask for each frame
- Save the final marked image with green contours overlaid on the enhanced frame
- Organize all outputs into clearly labelled folders by degradation category

6.2 Final Deployment Code

The integrated program has the following structure:

- Main runnable file: main.py (or equivalent)
- Separate module for enhancement functions
- Separate module for segmentation functions
- Input folder: contains the raw or extracted video frames
- Output folders: Enhanced_Frames/, Segmentation_Masks/, Final_Marked_Detections/
- Required libraries: OpenCV (cv2), NumPy, os
- To run: python main.py (with input/output paths configured at the top of the file)

6.3 GitHub Update

The GitHub repository has been updated with all source code files, original videos, extracted frames, enhanced frames, segmentation masks, final marked outputs, and sample result videos. A README file is included with instructions for running the integrated program.

GitHub Repository Link:

https://github.com/ayeshima/automated_road_condition_monitoring_system.git